

CUDA. Laboratorium 2.

Na tym laboratorium sprawdzamy na najprostszych algorytmach korzyści wynikające z użycia pamięci współdzielonej (*shared memory*) a nie tylko globalnej.

W dalszym ciągu korzystamy ze środowiska google colab. Jego przygotowanie jest takie jak w laboratorium 1 i tutaj tego nie powtarzam.

1. Najprostszy algorytm z wykorzystaniem pamięci globalnej i współdzielonej

W tym ćwiczeniu, uruchamiamy bardzo prosty algorytm, który wykonuje transpozycję macierzy kwadratowej. Do materiałów ćwiczenia dołączone są dwa programy: `matrix_transpose.cu` i `conflict_solved.cu`. W obu znajdują się po dwa kerneli, z których jeden wykorzystuje tylko pamięć globalną, a drugi dodatkowo pamięć współdzieloną. Kernel z pamięcią globalną jest identyczny w obu programach.

Proszę zwrócić uwagę, że w pierwszym kernelu, `matrix_transpose_naive` (tym z pamięcią globalną), zawsze mamy problem z dostępem łączonym (*coalesced access*) do pamięci globalnej. Jeżeli odczyt macierzy jest łączony (kolejne wątki odwołują się do kolejnych słów pamięci) to zapis macierzy transponowanej, nie jest. I odwrotnie, przy łączonym zapisie, odczyt nie może być taki.

W drugim kernelu, `matrix_transpose_shared`, zarówno odczyt jak i zapis z/do pamięci globalnej może być łączony dzięki temu, że transpozycja odbywa się w pamięci współdzielonej.

Drugi z programów, ten o nazwie `conflict_solved.cu` dodatkowo eliminuje tzw. *bank conflict* przy dostępie do pamięci współdzielonej. W architekturach równoległych (np. CUDA) pamięć współdzielona (ang. *shared memory*) jest podzielona na równe moduły (banki). Wątki uzyskują dostęp do pamięci jednocześnie podczas jednego cyklu zegara, o ile odczytywane lub zapisywane adresy znajdują się w różnych bankach. Jeśli kilka wątków próbuje dotrzeć do różnych adresów znajdujących się w *tym samym banku*, dostęp jest serializowany (wykonywany jeden po drugim), co drastycznie obniża wydajność. Proszę zwrócić uwagę jak *bank conflict* jest tu eliminowany.

1. Proszę wykonać program `matrix_transpose.cu` dla trzech różnych rozmiarów macierzy (np. 1024, 2048, 4096) i korzystając z wybranego modułu pomiaru czasu, zanotować czasy wykonania obu kerneli (global memory i shared memory bez rozwiązanego konfliktu banków). Dla większej stabilności wyników proszę uśrednić pomiar czasu z 10..20 powtórzeń, tak jak w ćwiczeniu z poprzedniego laboratorium
2. Proszę wykonać program `conflict_solved.cu` dla trzech różnych rozmiarów macierzy (takich jak wyżej) i korzystając z wybranego modułu pomiaru czasu, zanotować czasy wykonania obu kerneli (global memory i shared memory z rozwiązaniem konfliktu banków). Czas wykonania kernela z pamięcią globalną powinien być taki jak w punkcie 1. Dla

większej stabilności wyników proszę uśrednić pomiar czasu z 10..20 powtórzeń, tak jak w ćwiczeniu z poprzedniego laboratorium

3. Proszę zmodyfikować dowolny z powyższych programów, tak żeby w kernelu z pamięcią globalną, zamiast wykonywać transpozycję, jedynie przepisywał macierz wejściową do wyjściowej. Trochę bez sensu, ale proszę zanotować czas gdy dostęp do pamięci globalnej jest łączony (coalesced).

4. Proszę dodatkowo uruchomić każdy z programów pod kontrolą profilera `ncu` :

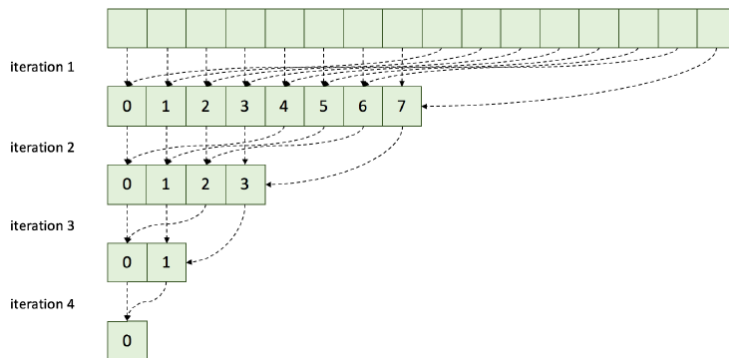
```
!ncu ./matrix_transpose
```

Z wielu licznych wartości jakie się pojawią, proszę zanotować wartość parametrów z grupy Occupancy. **Occupancy** (wypełnienie/zajętość) w CUDA jest to stosunek liczby aktywnych wątków (lub warpów) wykonywanych na jednym multiprocesorze strumieniowym (SM) do maksymalnej liczby wątków, jaką ten procesor może jednocześnie obsłużyć. Pozwala ocenić, jak efektywnie wykorzystywane są zasoby obliczeniowe karty graficznej. Proszę zwrócić uwagę na `Achieved Occupancy` i `Achieved Active Warps Per SM`

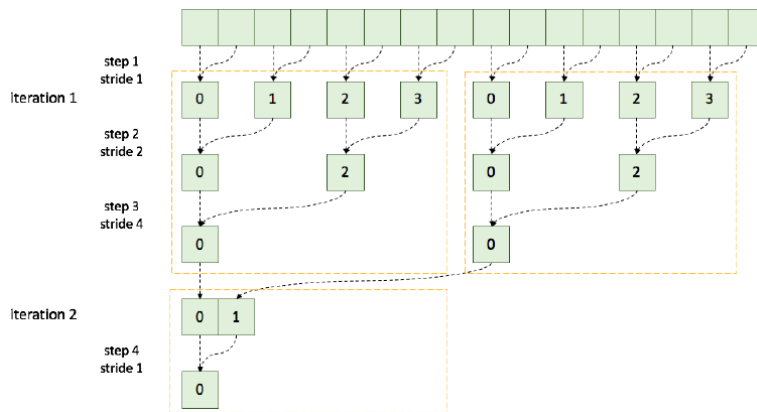
5. Wyniki proszę zapisać w tabelce.

6. Redukcja.

Zadaniem jest zsumowanie elementów wektora. W przykładowym programie zaproponowane są dwa podejścia. Pierwsze, określane naiwnym, działa na pamięci globalnej i sumuje parami elementy jak przedstawiono na rysunku:



Drugie podejście opiera się na pamięci współdzielonej, której użycie ilustruje rysunek. Częściowe sumowanie odbywa się w pamięci współdzielonej. Operacje w jednym bloku na pamięci współdzielonej zaznaczone prostokątem z pomarańczową linią przerywaną.



W ramach ćwiczenia proszę na podstawie załączonych plików skompilować i uruchomić dwa programy `reduction_global` i `reduction_shared`. Proszę przedstawić porównanie czasów wykonania obiema metodami dla zadanego rozmiaru wektora wejściowego.

7. Histogram i operacje atomowe – zadanie dodatkowe

To jest wstępne ćwiczenie dotyczące histogramu. Ma na celu pokazać przykład wykorzystania operacji atomowych. Proszę porównać czasy i wyniki w dwóch podejściach zastosowanych do liczenia histogramu w programie `histo.cu`. Pierwsze podejście sumuje elementy histogramu wprost w pamięci globalnej, nie patrząc na niechciane interakcje między wątkami. Druga separuje od siebie obliczenia poszczególnych wątków poprzez użycie operacji atomowej..

Proszę dokonać obliczeń (czasów i wyników numerycznych) dla jednego długiego wektora danych wejściowych (w przykładzie jest domyślna wartość `ARRAY_SIZE = 65536`, można ją powiększyć) i dwóch rozmiarów histogramu, np. dla domyślnej wartości `BIN_COUNT = 16`, i innej istotnie większej, np. 32768).

Proszę krótko skomentować otrzymane wyniki.